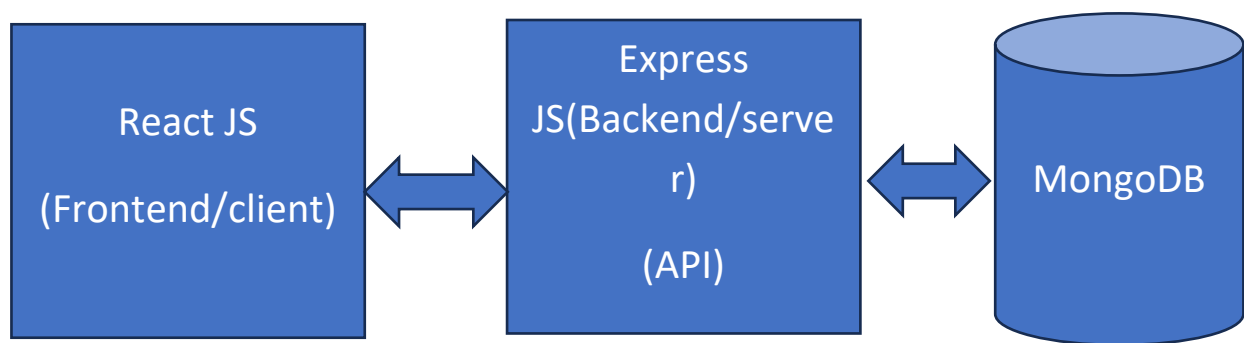


MERN Stack

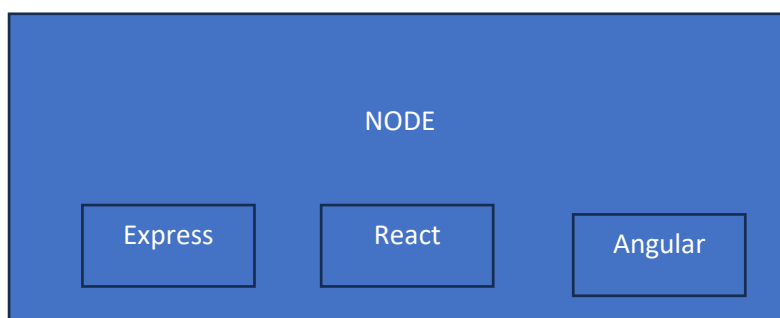
The MERN stack is a widely adopted full-stack development framework that simplifies the creation of modern(dynamic) web applications. Using JavaScript for both the frontend and backend enables developers to efficiently build robust, scalable, and dynamic applications.

I need a package to run ReactJS, ExpressJS, it is known as Node



CRUD Operations

(Create, Read, Update, Delete)



What is MERN Stack?

MERN Stack is a JavaScript Stack that is used for easier and faster deployment of full-stack web applications. MERN Stack comprises of 4 technologies namely: MongoDB, Express, React and NodeJS. It is designed to make the development process smoother and easier.

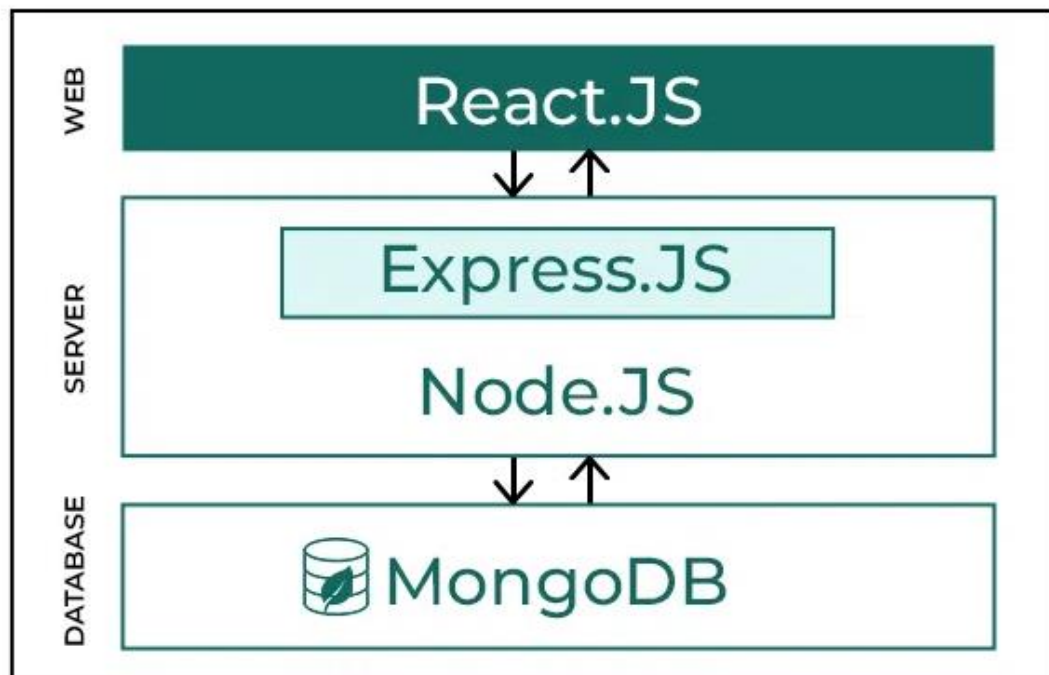
- **MongoDB:** Non-Relational(NoSQL) Database
- **Express:** NodeJS web server
- **React:** JavaScript Frontend Framework
- **Node:** JavaScript Web Server

How MERN Stack Works?

The MERN stack is a popular JavaScript-based web development framework for building modern web applications. It consists of four key technologies working together:

- **MongoDB (Database):** MongoDB is a NoSQL document database used to store the application's data. It's flexible and scalable, ideal for handling large amounts of information.
- **ExpressJS (Backend Framework):** ExpressJS is a NodeJS web application framework that provides tools for creating APIs and handling HTTP requests. It acts as the server-side logic of the application.
- **React (Frontend Library):** React is a JavaScript library for building user interfaces (UIs). It's used to create dynamic and interactive front-end components of the website that users interact with.
- **NodeJS (Runtime Environment):** NodeJS is a JavaScript runtime environment that allows you to run JavaScript code on the server. It's what powers the backend (ExpressJS) of the application.

How MERN Stack Works?



Getting Started with MERN Stack

There are two requirements to start a MERN stack project:

Step 1: Install node on your system depending on your Operating System:

[Installation of NodeJS on Windows](#)

Step 2: You need a code editor to work with so install a code editor, preferably we will use VS Code

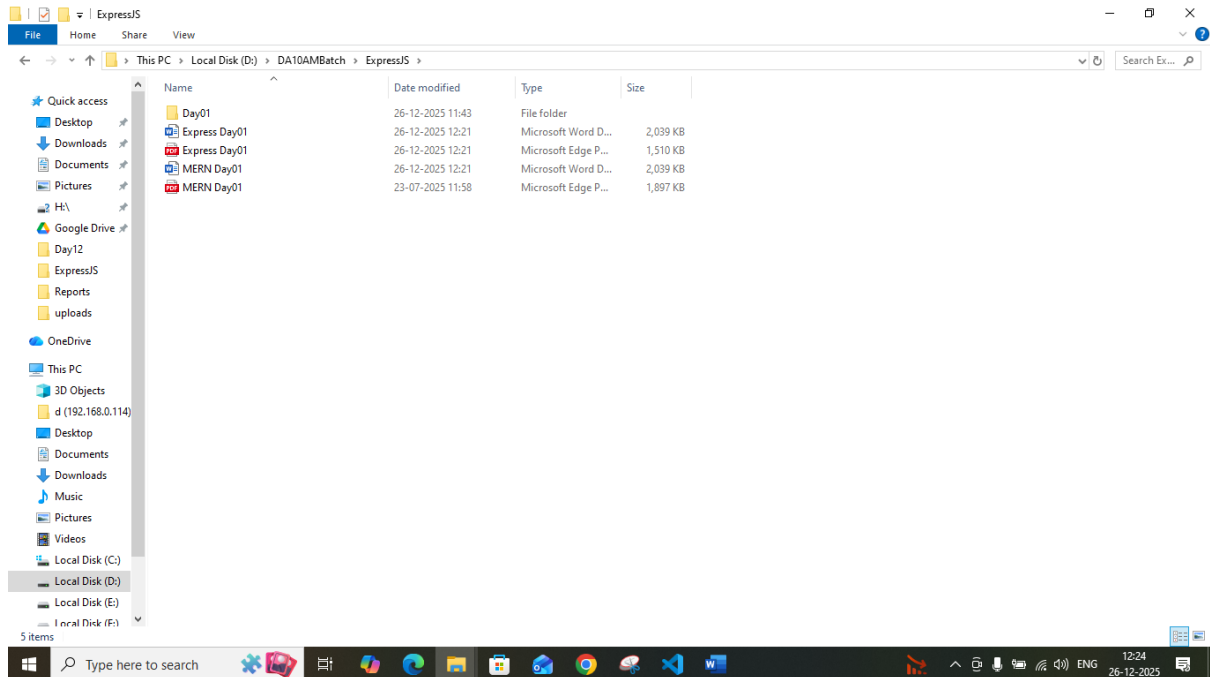
Setting Up a MERN Stack Project

To setup MERN stack we need to create a folder structure for both frontend and backend. Then we have to define database schema to store and retrieve data from the database.

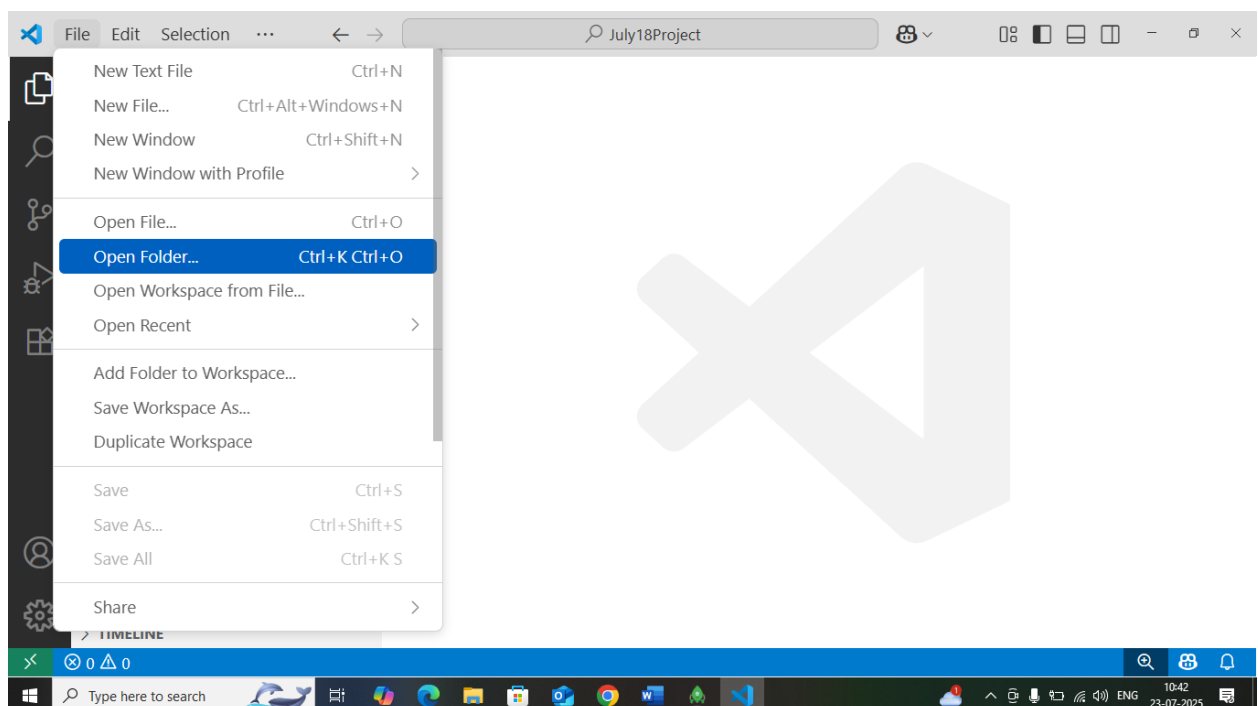
Follow the below steps to create a basic structure

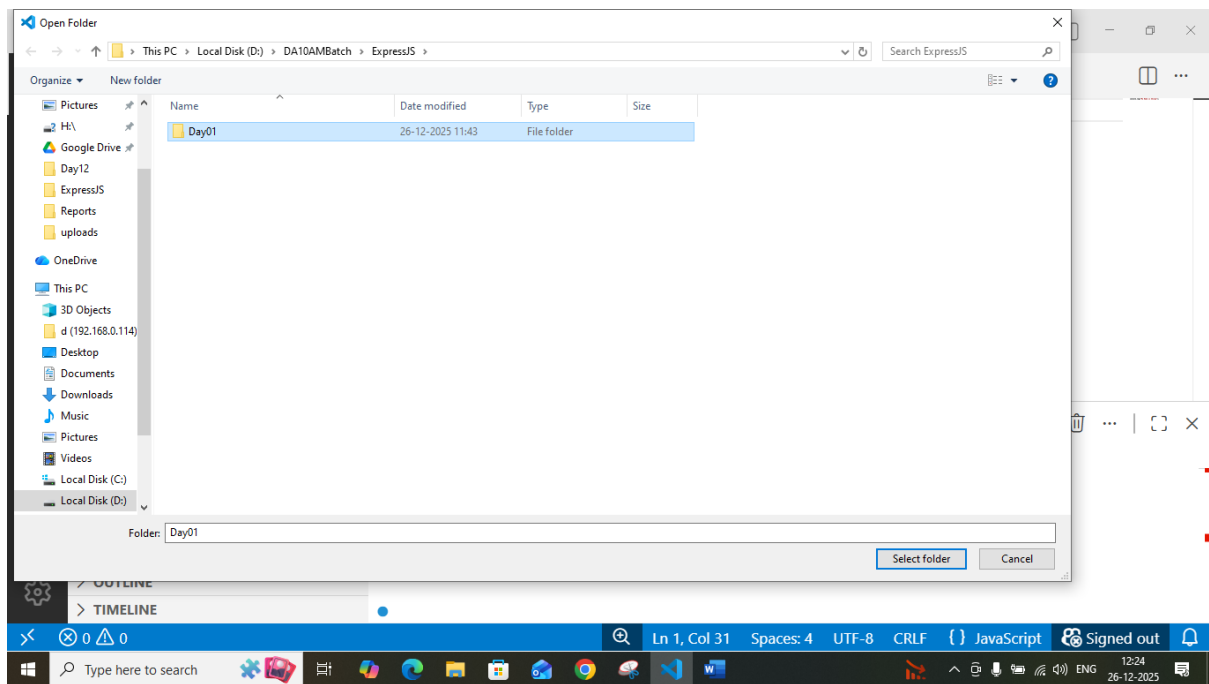
Step 1: After the code editor is installed, create a new project folder. Then go to the project folder in command prompt/terminal and type below commands to create folder for frontend and backend

Go to any drive & create a folder => Day01



Open that folder in VSCode





2. Express: Back-End Framework

Express is a NodeJS framework. Rather than writing the code using NodeJS and creating loads of Node modules, Express makes it simpler and easier to write the back-end code. Express helps in designing great web applications and APIs. Express supports many middlewares which makes the code shorter and easier to write.

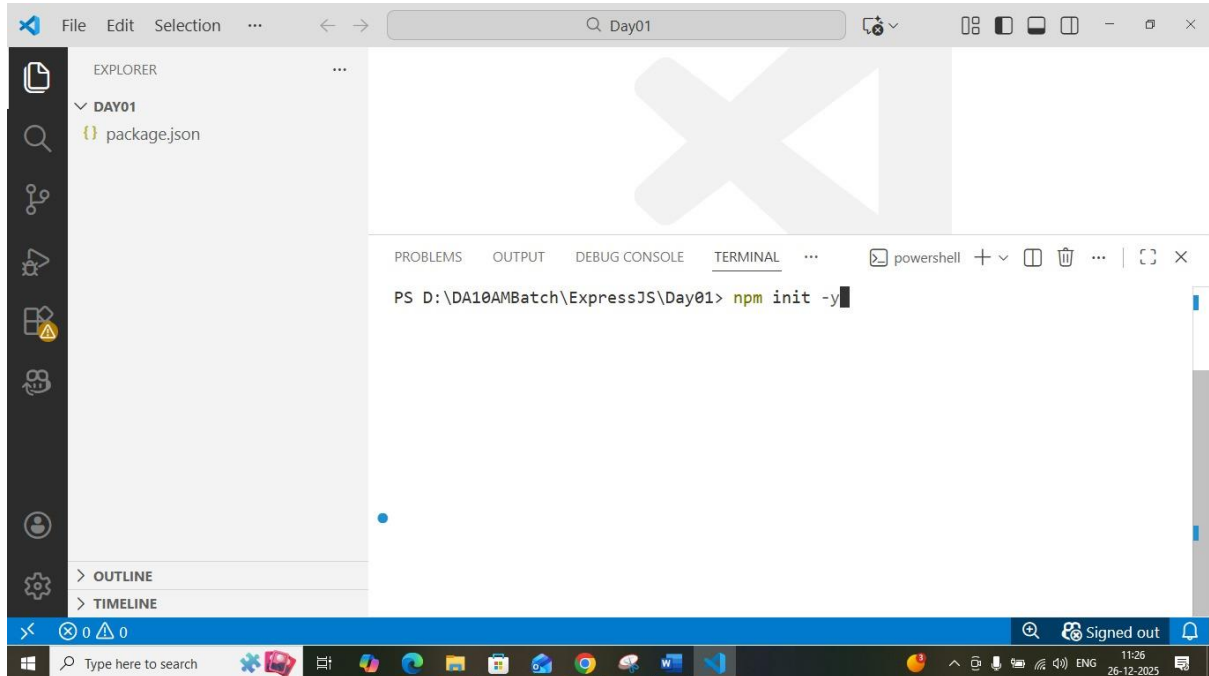
Why use Express?

- Asynchronous and Single-threaded.
- Efficient, fast & scalable
- Has the biggest community for NodeJS
- Express promotes code reusability with its built-in router.
- Robust API
- Create a new folder to start your express project and type below command in the command prompt to initialize a package.json file.
Accept the default settings and continue.

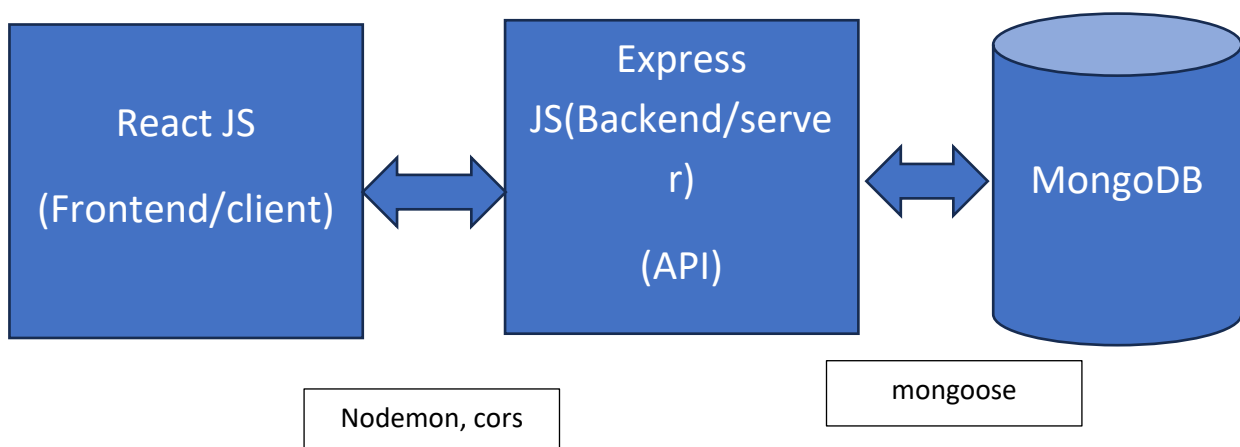
Step 5: Initialize the project backend using the command

```
npm init -y
```

By using this command package.json file will get created



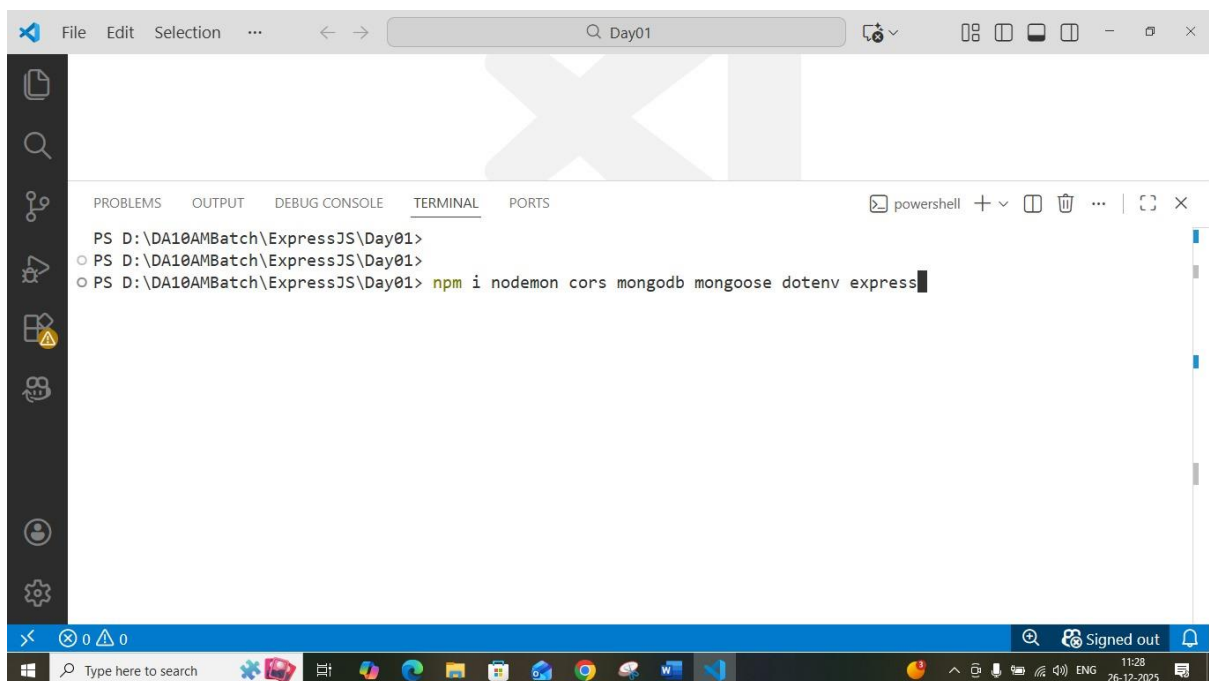
- Then install express by typing the below command and hit enter. Now finally create a file inside the directory named index.js.



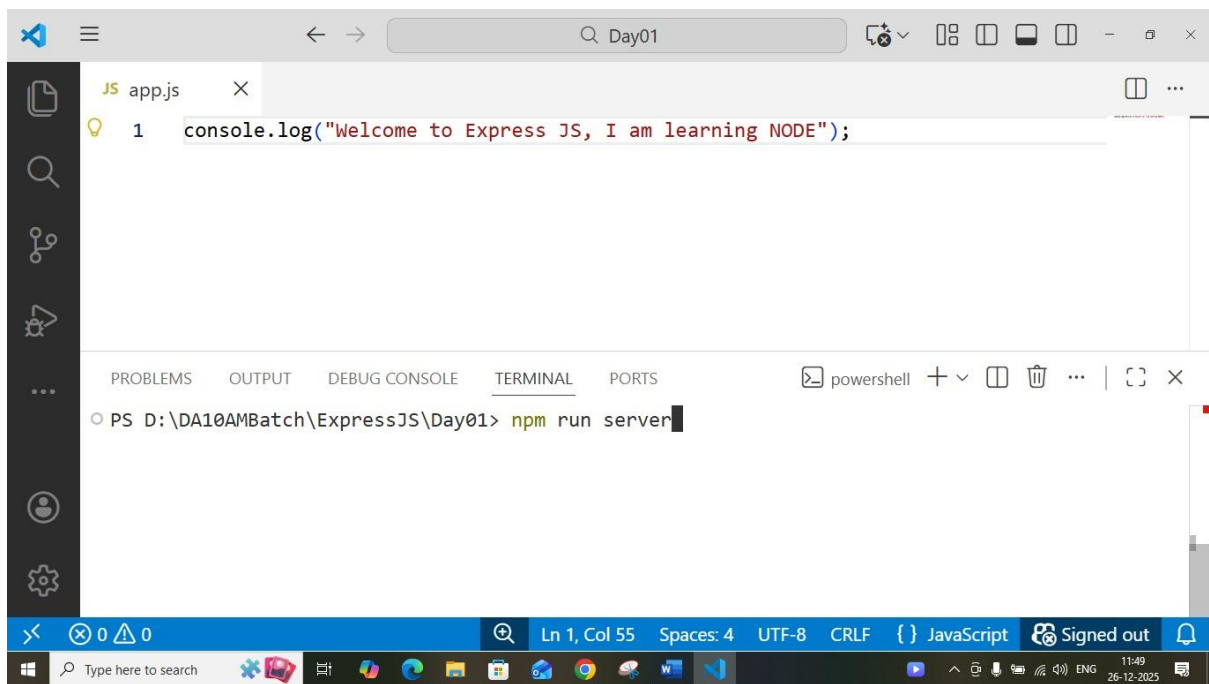
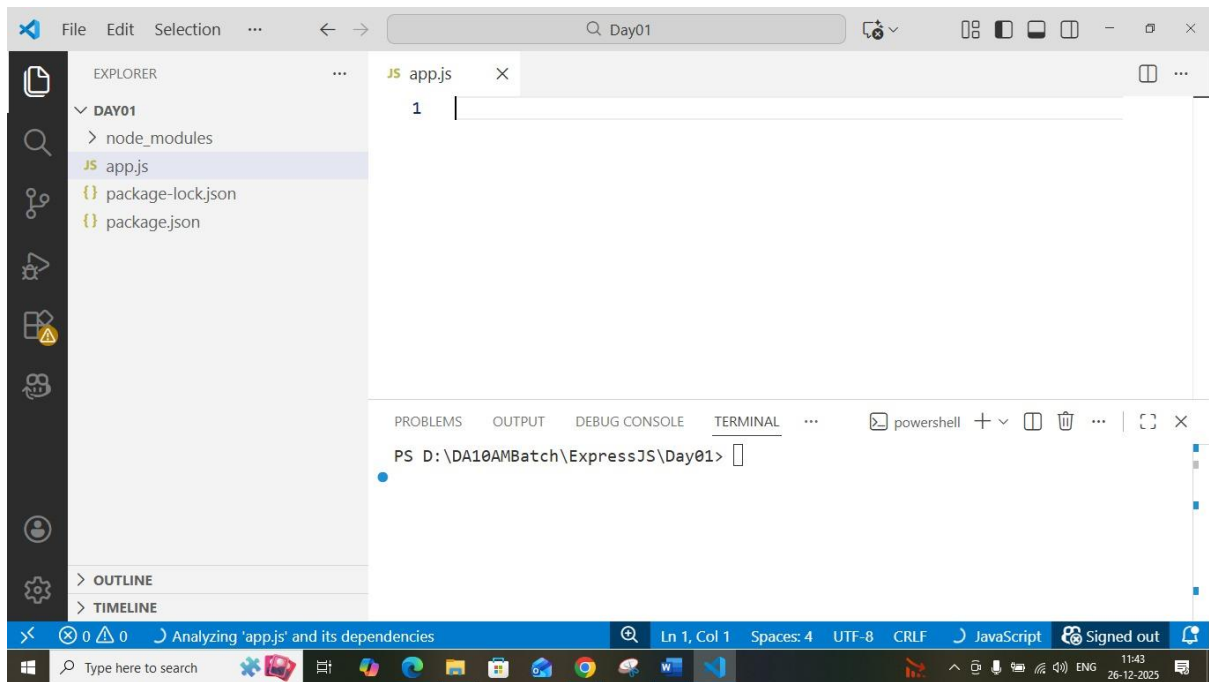
Step 6: Install Express and other backend dependencies using the command

```
npm i mongodb express cors dotenv nodemon  
mongoose
```

when we execute the above command automatically a folder will get created called **node_modules** inside this folder all the libraries will get stored

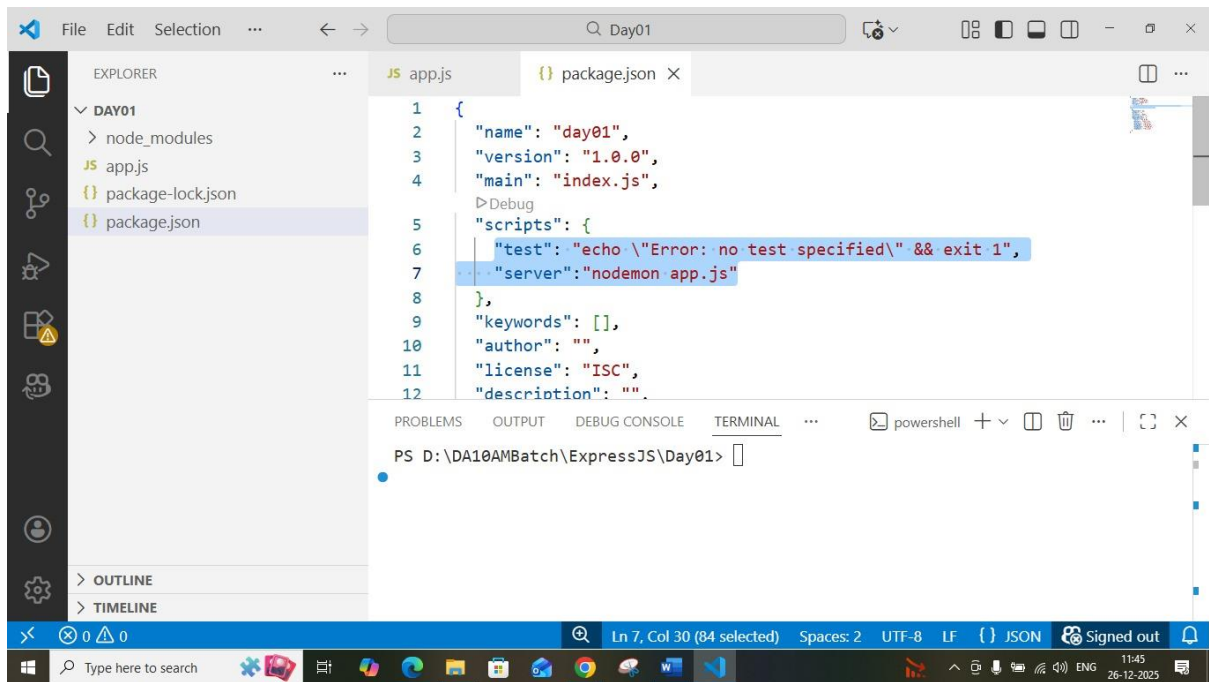


Create a file in Day01 => app.js



=> app.js

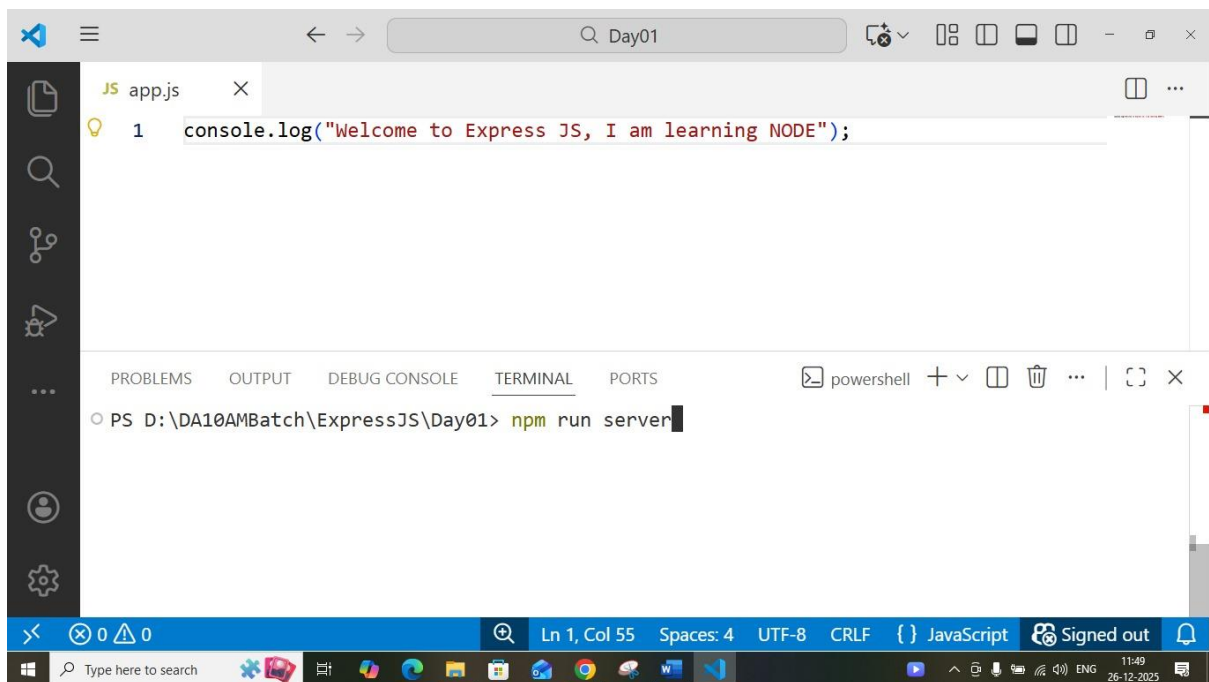
Set up the package.json to run the above file



This screenshot shows the Visual Studio Code interface with the `package.json` file open. The Explorer sidebar on the left shows the project structure for `DAY01`, including `node_modules`, `app.js`, `package-lock.json`, and `package.json`. The `package.json` file contains the following JSON:

```
1 {
2   "name": "day01",
3   "version": "1.0.0",
4   "main": "index.js",
5   "scripts": {
6     "test": "echo \"Error: no test specified\" && exit 1",
7     "server": "nodemon app.js"
8   },
9   "keywords": [],
10  "author": "",
11  "license": "ISC",
12  "description": ""
13 }
```

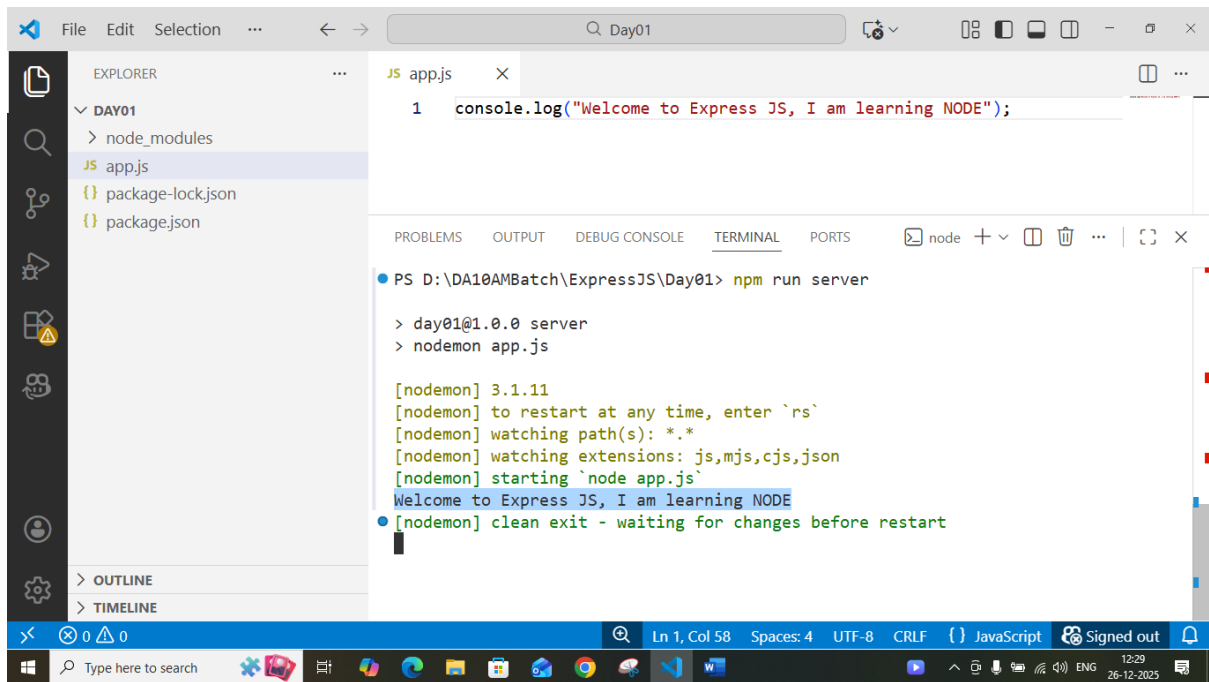
The terminal at the bottom shows the PowerShell prompt: `PS D:\DA10AMBatch\ExpressJS\Day01>`. The status bar at the bottom indicates the file is in JSON format, signed out, and the system time is 11:45 on 26-12-2025.



This screenshot shows the Visual Studio Code interface with the `app.js` file open. The Explorer sidebar on the left shows the project structure for `DAY01`, including `node_modules`, `app.js`, `package-lock.json`, and `package.json`. The `app.js` file contains the following JavaScript code:

```
1 console.log("Welcome to Express JS, I am learning NODE");
```

The terminal at the bottom shows the PowerShell prompt: `PS D:\DA10AMBatch\ExpressJS\Day01> npm run server`. The status bar at the bottom indicates the file is in JavaScript format, signed out, and the system time is 11:49 on 26-12-2025.



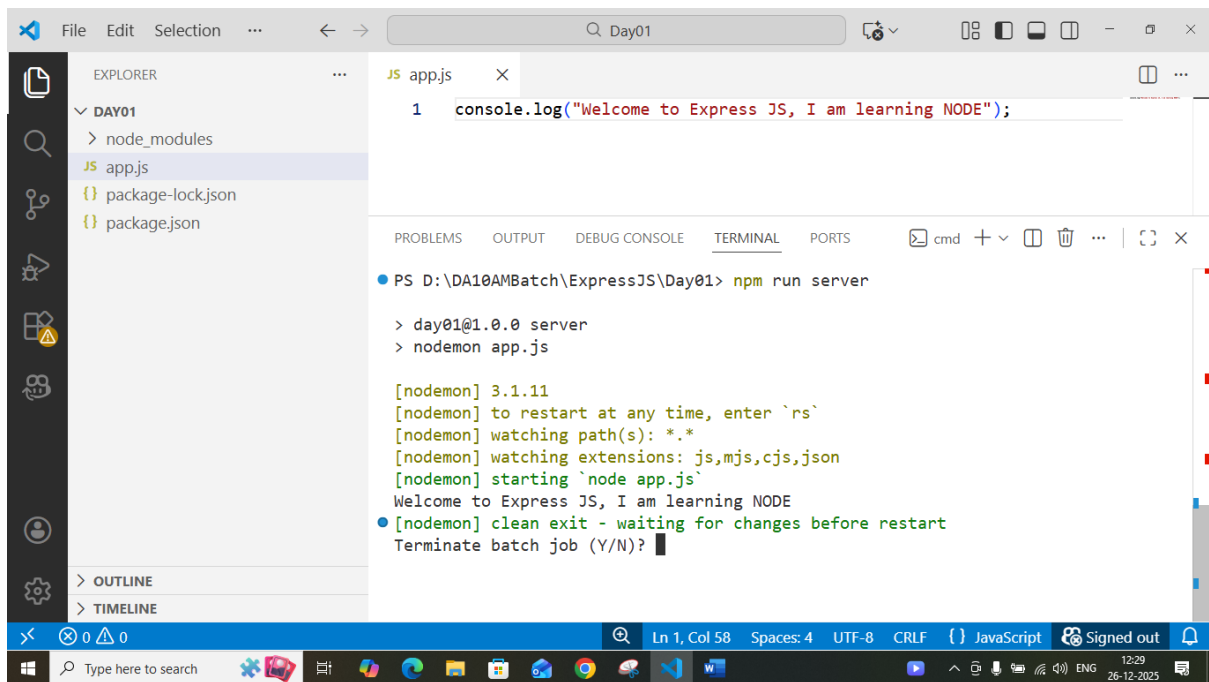
The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left displays the file structure for a project named 'DAY01', which includes 'node_modules', 'app.js', 'package-lock.json', and 'package.json'. The main editor window is open to 'app.js' and contains a single line of code: `1 console.log("Welcome to Express JS, I am learning NODE");`. The bottom panel is divided into 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', and 'TERMINAL' tabs. The 'TERMINAL' tab is active, showing the command prompt output for running the server. The commands entered are `npm run server` and `nodemon app.js`. The output shows the server starting successfully with the message 'Welcome to Express JS, I am learning NODE'.

```
PS D:\DA10AMBatch\ExpressJS\Day01> npm run server

> day01@1.0.0 server
> nodemon app.js

[nodemon] 3.1.11
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
Welcome to Express JS, I am learning NODE
[nodemon] clean exit - waiting for changes before restart
```

Press ctrl+c to stop the server



This screenshot shows the same VS Code interface as the previous one, but the terminal output now includes the command to stop the server. The user has entered `ctrl+c` in the terminal, which has resulted in the message 'Terminate batch job (Y/N)?' appearing at the end of the output. The rest of the terminal output is identical to the previous screenshot.

```
PS D:\DA10AMBatch\ExpressJS\Day01> npm run server

> day01@1.0.0 server
> nodemon app.js

[nodemon] 3.1.11
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
Welcome to Express JS, I am learning NODE
[nodemon] clean exit - waiting for changes before restart
Terminate batch job (Y/N)?
```

